

Chapter 3

Semantic Segmentation and Style Transfer

In the previous chapter, we explored object detection, where models predict coarse spatial locations using bounding boxes. While bounding boxes are sufficient for many real-world applications, many tasks require a far higher degree of granularity. Medical imaging, autonomous scene parsing, and robotic manipulation demand an understanding of visual scenes down to the individual pixel.

This chapter shifts our focus from coarse region proposals to dense, pixel-level predictions and visual synthesis. We begin with semantic segmentation, exploring how Fully Convolutional Networks (FCNs) and U-Net architectures achieve pixel-wise classification. Central to this is the mathematical mechanism of transposed convolutions and specialized spatial upsampling operators. Moving beyond standard classification, we then explore neural style transfer, demonstrating how intermediate activations of deep CNNs can isolate and recombine the structural content of one image with the artistic texture of another using Gram matrix feature representations and multi-objective optimization. Finally, we situate these concepts within the broader ecosystem of advanced computer vision tasks, including instance segmentation, panoptic segmentation, and keypoint estimation.

3.1 Semantic Segmentation

The problem of **semantic segmentation** focuses on how to divide an image into regions belonging to different semantic classes. Different from object detection, semantic segmentation recognizes and understands what are in images in pixel level, as labeling is at pixel level. Figure 3.1 shows an example of semantic segmentation. Compared with object detection, the pixel-level borders labeled in semantic segmentation are obviously more fine-grained.

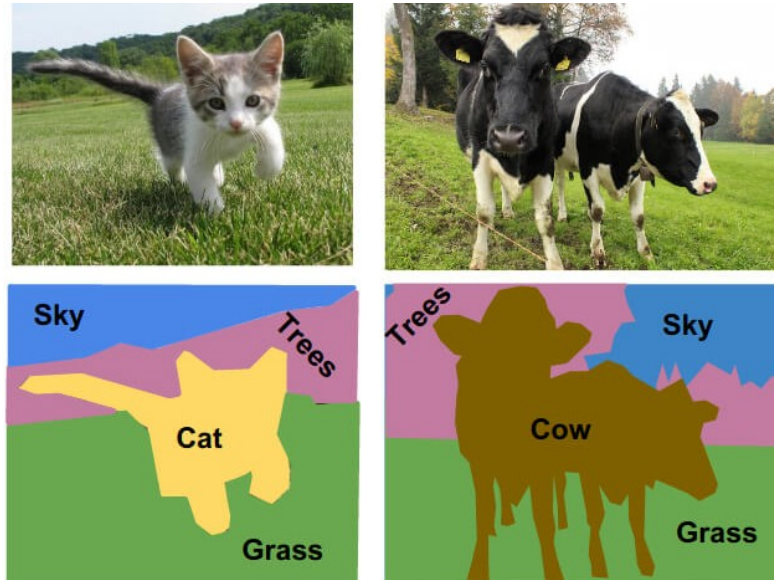


Figure 3.1: Semantic segmentation of two different images.

3.1.1 Naive Approach

A natural and naive approach is to use a **sliding window**. We would divide the image into a grid of patches, apply a CNN to each one and classify the center pixel of each patch. Putting all the pixels together, we would obtain a new image with all the pixels classified. However, this approach is not very practical because many forward passes would have to be done and that is unefficient. Also, we would not reuse shared features between overlapping patches.

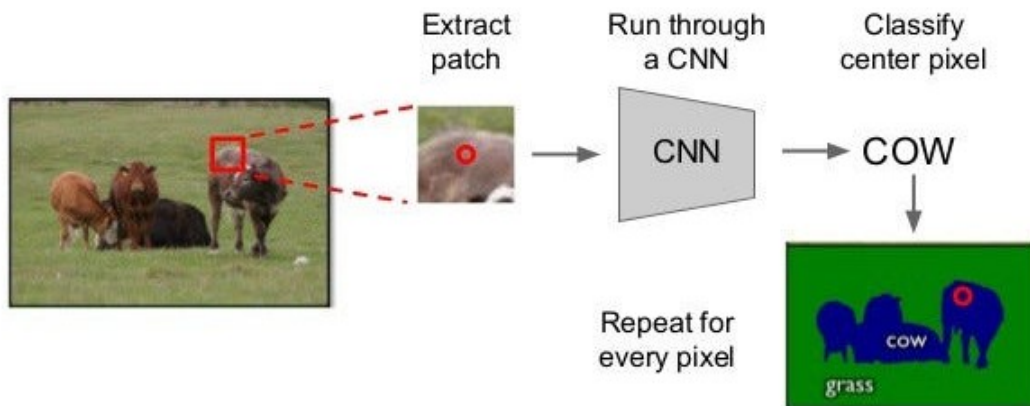


Figure 3.2: Naive approach for semantic segmentation.

3.1.2 Transposed Convolution

The CNN layers, such as convolutional layers and pooling layers, typically reduce (downsample) the spatial dimensions (height and width) of the input, or keep them unchanged. In semantic segmentation, which classifies at pixel-level, it would be convenient if the spatial

dimensions of the input and output are the same. By doing this, the channel dimension at one output pixel can hold the classification results for the input pixel at the same spatial position.

Example: Semantic segmentation

Suppose we have an image and there are two classes: “cat” and “background”. We want to classify all pixels. Let’s focus on the pixel of the upper right corner. If we are able to obtain the same spatial dimensions (height and width) on the output, for example using padding, we can force the output of the CNN to have two channels, one for each class, and we can apply softmax along the channels dimension to interpret the value of the upper right corner of each channel to be the probability of that pixel belonging to the corresponding class. The same logic can be applied to all pixels. You can visualize this example in Figure 3.3.

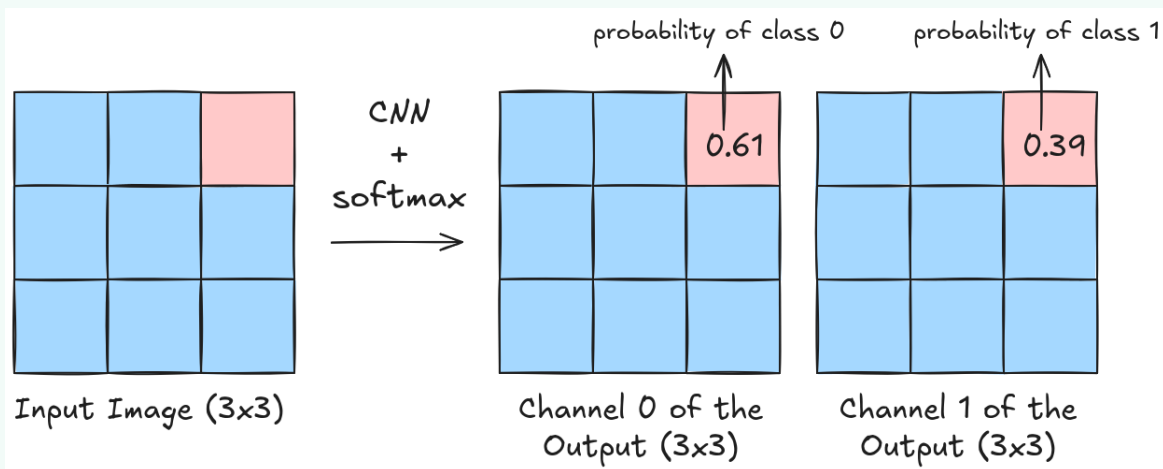


Figure 3.3: Example of classifying one pixel. Note that $0.61 + 0.39 = 1$.

To achieve this, especially after the spatial dimensions are reduced by CNN layers, we can use another type of CNN layers that can **increase (upsample)** the spatial dimensions of intermediate feature maps: **transposed convolutions**.

Basic Operation

Ignoring channels for now, let’s begin with the basic transposed convolution operation with stride of 1 and no padding. Suppose that we are given a $n_h \times n_w$ input tensor and a $k_h \times k_w$ kernel. Sliding the kernel window with stride of 1 for n_w times in each row and n_h times in each column yields a total of $n_h \cdot n_w$ intermediate results. Each intermediate result is a $(n_h + k_h - 1) \times (n_w + k_w - 1)$ tensor initialized as zeros. To compute each intermediate tensor, each element in the input tensor is multiplied by the kernel so that the resulting $k_h \times k_w$ tensor replaces a portion in each intermediate tensor. Note that the position of the replaced portion in each intermediate tensor corresponds to the position of the element in the input tensor used for the computation. In the end, all the intermediate results are summed over

to produce the output.

Example: Transposed convolution

We will illustrate how transposed convolution with a 2×2 kernel is computed for a 2×2 input tensor. Let's say the input and the kernel are respectively

$$\begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix} \text{ and } \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix}.$$

Then the transposed convolution (denoted as $*^{-1}$) is

$$\begin{aligned} \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix} *^{-1} \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix} &= 0 \begin{pmatrix} 0 & 1 & - \\ 2 & 3 & - \\ - & - & - \end{pmatrix} + 1 \begin{pmatrix} - & 0 & 1 \\ - & 2 & 3 \\ - & - & - \end{pmatrix} + 2 \begin{pmatrix} - & - & - \\ 0 & 1 & - \\ 2 & 3 & - \end{pmatrix} + 3 \begin{pmatrix} - & - & - \\ - & 0 & 1 \\ - & 2 & 3 \end{pmatrix} \\ &= \begin{pmatrix} 0 & 0 & - \\ 0 & 0 & - \\ - & - & - \end{pmatrix} + \begin{pmatrix} - & 0 & 1 \\ - & 2 & 3 \\ - & - & - \end{pmatrix} + \begin{pmatrix} - & - & - \\ 0 & 2 & - \\ 4 & 6 & - \end{pmatrix} + \begin{pmatrix} - & - & - \\ - & 0 & 3 \\ - & 6 & 9 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 4 & 6 \\ 4 & 12 & 9 \end{pmatrix}. \end{aligned}$$

Padding, Strides and Multiple Channels

Different from in the regular convolution where **padding** is applied to input, it is **applied to output** in the transposed convolution. For example, when specifying the padding number on either side of the height and width as 1, the first and last rows and columns will be removed from the transposed convolution output. In general, if the padding is p , then $2p$ (first and last p) rows and columns will be removed.

Example: Padding

If we use padding 1 to the previous example, we remove first and last rows and first and last columns. Therefore, the output of the transposed convolution will be

$$\begin{pmatrix} \emptyset & \emptyset & \cancel{1} \\ \emptyset & 4 & \emptyset \\ \cancel{4} & \cancel{12} & \emptyset \end{pmatrix} = (4).$$

In the transposed convolution, **strides are specified for intermediate results (thus output)**, not for input. Now, for stride s , each intermediate result is a

$$[(n_h - 1)s + (k_h - 1) + 1] \times [(n_w - 1)s + (k_w - 1) + 1]$$

tensor initialized as zeros. Note that this dimensions are true only if padding is equal to zero.

For multiple input and output **channels**, the transposed convolution works in the **same way as the regular convolution**. Suppose that the input has c channels, and that the transposed convolution assigns a $k_h \times k_w$ kernel tensor to each input channel. When multiple output channels are specified, we will have a $c \times k_h \times k_w$ kernel for each output channel.

Example: Stride

If we use the same example as before but with stride 2, then

$$\begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix} *^{-1} \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix} = 0 \begin{pmatrix} 0 & 1 & - & - \\ 2 & 3 & - & - \\ - & - & - & - \\ - & - & - & - \end{pmatrix} + 1 \begin{pmatrix} - & - & 0 & 1 \\ - & - & 2 & 3 \\ - & - & - & - \\ - & - & - & - \end{pmatrix} + 2 \begin{pmatrix} - & - & - & - \\ - & - & - & - \\ 0 & 1 & - & - \\ 2 & 3 & - & - \end{pmatrix} \\ + 3 \begin{pmatrix} - & - & - & - \\ - & - & - & - \\ - & - & 0 & 1 \\ - & - & 2 & 3 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 3 & 3 \\ 0 & 2 & 0 & 3 \\ 4 & 6 & 6 & 9 \end{pmatrix}.$$

Initializing Transposed Convolutional Layers

We could initialize transposed convolutional layers randomly, but there are more effective ways. The idea is to initialize the weights of the kernel in such a way that we obtain the same output as if we did an upsampling method. Most of these methods are traditional methods for upsampling images. Some of them are shown in Figure 3.4:

- **Bed of Nails:** Fills with zeros the new generated pixels.
- **Nearest Neighbor:** Fills with the value of the nearest neighbor the new generated pixels.
- **Bilinear Interpolation:** It is a more advanced but also complicated method. You can find more information about it in this [link](#).

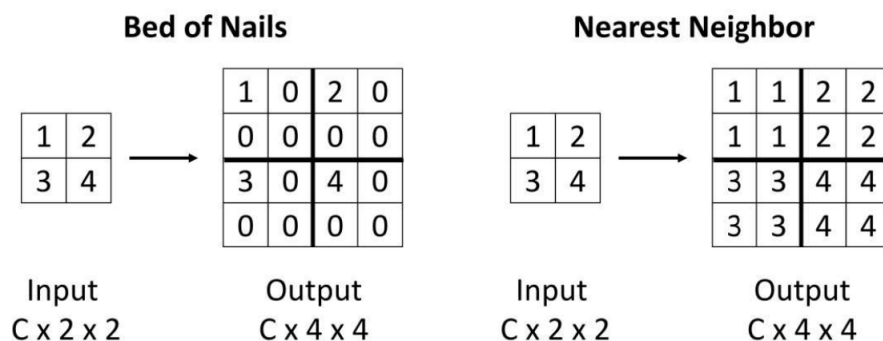


Figure 3.4: Example of bed of nails and nearest neighbor.

An alternative that can be used and has no parameters to learn is **max unpooling** (see Figure 3.5): It takes a value and places it at the original position where the maximum was during pooling, padding the remaining positions with zeros. It needs to know where each maximum was in the original input. Therefore, during max pooling, the indices of the

maximum positions are saved and then those indices are used to correctly locate the values during expansion.

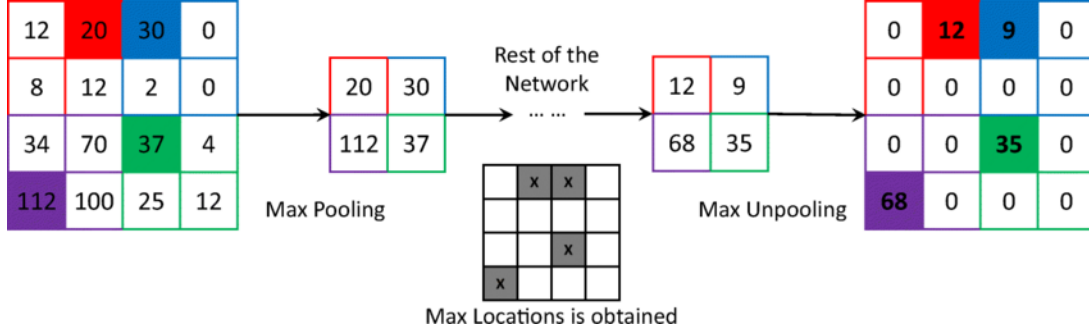


Figure 3.5: Example of max unpooling.

Example: Bed of nails and nearest neighbor

If we choose bed of nails and we want to upsample from 2×2 to 4×4 , we would use a stride of 2 and the initial weights of the kernel would be

$$\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}.$$

Also, if we choose nearest neighbor we would use a stride of 2 and the initial weights of the kernel would be

$$\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}.$$

Check exercises 5 and 6 to get a better understanding of the logic behind this.

3.1.3 Fully Convolutional Networks

A **fully convolutional network (FCN)** uses a convolutional neural network to transform image pixels to pixel classes. First, it applies a CNN to reduce the height and width of intermediate feature maps, then a 1×1 convolution with as many output channels as classes there are (each channel corresponds to a class), and finally uses transposed convolutions to go back to the same shape of the original image. Each pixel is assigned a value in each channel that corresponds to the probability of that pixel belonging to the class associated to that channel. As a result, the classification output and the input image have a one-to-one correspondence in pixel level. This is the idea previously seen in Figure 3.3.

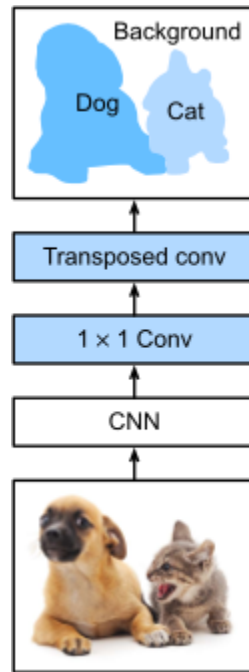


Figure 3.6: Fully convolutional network.

Initially, the convolutions used a 1×1 kernel or padding to not modify the dimensions of the image and finally a convolutional layer with as many channels as classes there were. However, this was unefficient, and therefore the dimensions were reduced at first and then upsampled to the original ones using the transposed convolutions. In both approaches the loss function used to train is the cross entropy.

3.1.4 U-Net

A particular case of the fully convolutional networks is the **U-Net**, shown in Figure 3.7, whose architecture is designed for image segmentation tasks, especially in the biomedical field. Its structure is U-shaped because it combines a contraction path (encoder) and an expansion path (decoder). In the contraction phase, the network applies successive convolutions and max pooling operations to capture features and progressively reduce the spatial resolution of the image, thus extracting increasingly abstract representations. In the expansion phase, it uses transposed convolutions to increase spatial resolution to the original one and reconstruct the segmentation map.

A key feature of U-Net are **skip connections**, which are long encoder-to-decoder feature transfers, which link the encoder layers with the corresponding decoder layers. In practice, this is implemented by concatenating the activation maps of the encoder with those of the decoder at the same resolution level. This way, the decoder can take advantage of both the global information extracted by the encoder and the original local details, allowing for the reconstruction of accurate segmentation maps with well-defined contours. Thanks to this architecture, U-Net can produce accurate segmentations even with limited training data.

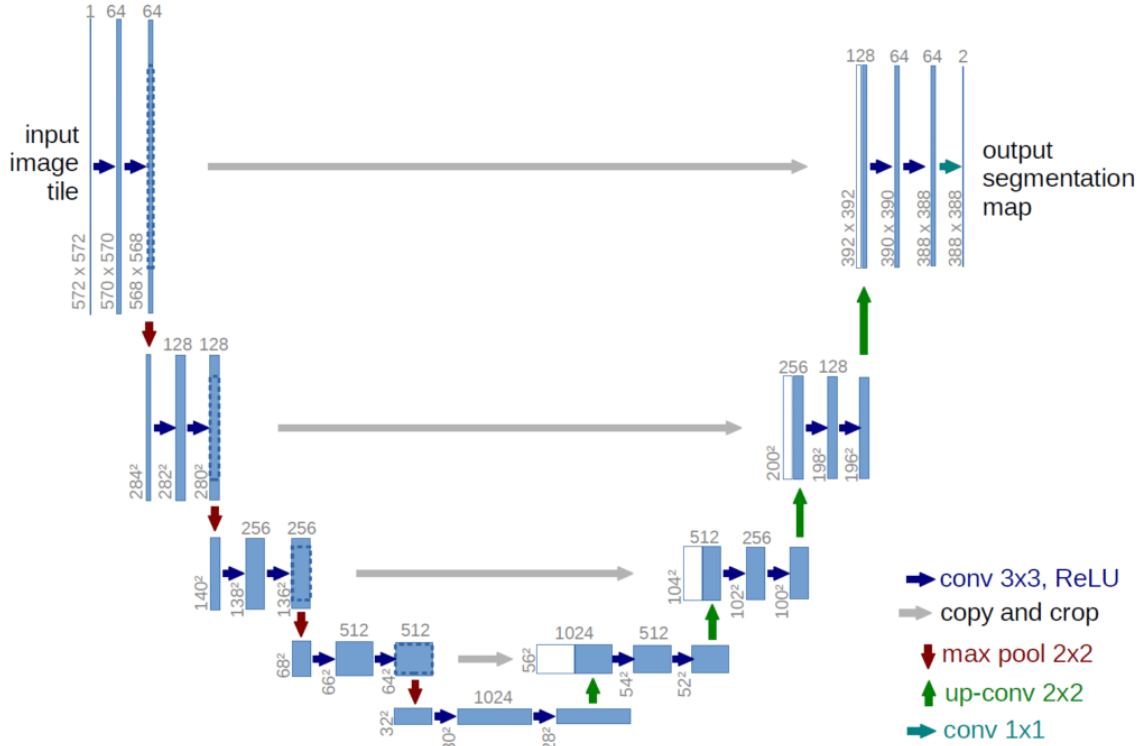


Figure 3.7: Architecture of the U-Net. The dimensions and number of layers may vary, but the U structure and the skip connections are always the same. Also, note that the output size is smaller than the original one, but it can be upscaled.

Note

The skip connections are same called as in ResNet, but although the idea is similar, here the implementation is a bit different.

3.2 Style Transfer

In this section, we will leverage layerwise representations of a CNN to automatically apply the style of one image to another image, i.e., **style transfer**. This task needs two input images: one is the **content image** and the other is the **style image**. We will use neural networks to modify the content image to make it close to the style image in style. For example, in Figure 3.8 you can see how the content of the Mona Lisa is combined with the style of a Picasso painting to create a new synthesized version.

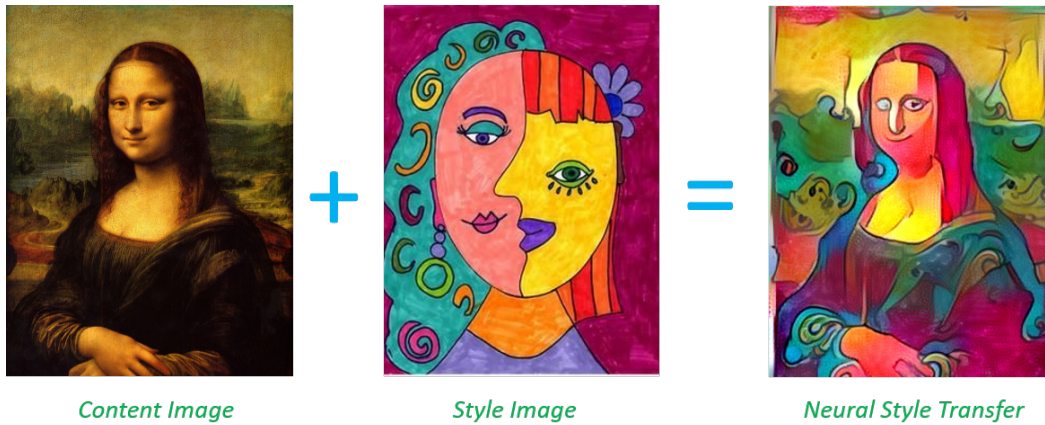


Figure 3.8: Given content and style images, style transfer outputs a synthesized image.

3.2.1 Method

First, we initialize the synthesized image as the content image. The synthesized image is the only variable that needs to be updated during the style transfer process (content and style image are always the same), i.e., **the only parameters to be updated during training are the pixels of the synthesized image**. Then we choose a pretrained CNN and freeze its parameters; we will use it to extract image features. We can choose the output of some of these layers as content features or style features. Take Figure 3.9 as an example. The pretrained neural network here has 3 convolutional layers, but we only keep the second layer outputs for the content features, and only the the first and third layers outputs for style features. You can think of the layers we choose as an hyperparameter. Note that the pretrained network is the same for content, synthesized and style images and its parameters will be frozen, as the only parameters to be updated are the pixels of the synthesized image, which do not belong to the pretrained network.

Note

The synthesized image is usually initialized as the content image, but it can also be initialized as random noise, as the style image or a weighted average (interpolation) between the content image x^C and the style image x^S : $\alpha x^C + (1 - \alpha)x^S$ with $\alpha \in [0, 1]$.

Next, we calculate the loss function of style transfer through forward propagation (direction of solid arrows), and update the trainable parameters (the synthesized image) through back-propagation (direction of dashed arrows). The loss function commonly used in style transfer consists of three parts:

- **Content loss:** Makes the synthesized image and the content image close in content features
- **Style loss:** Makes the synthesized image and style image close in style features.
- **Total variation loss:** Helps to reduce the noise in the synthesized image.

Finally, when the model training is over, we output the trainable parameters of the style transfer to generate the final synthesized image.

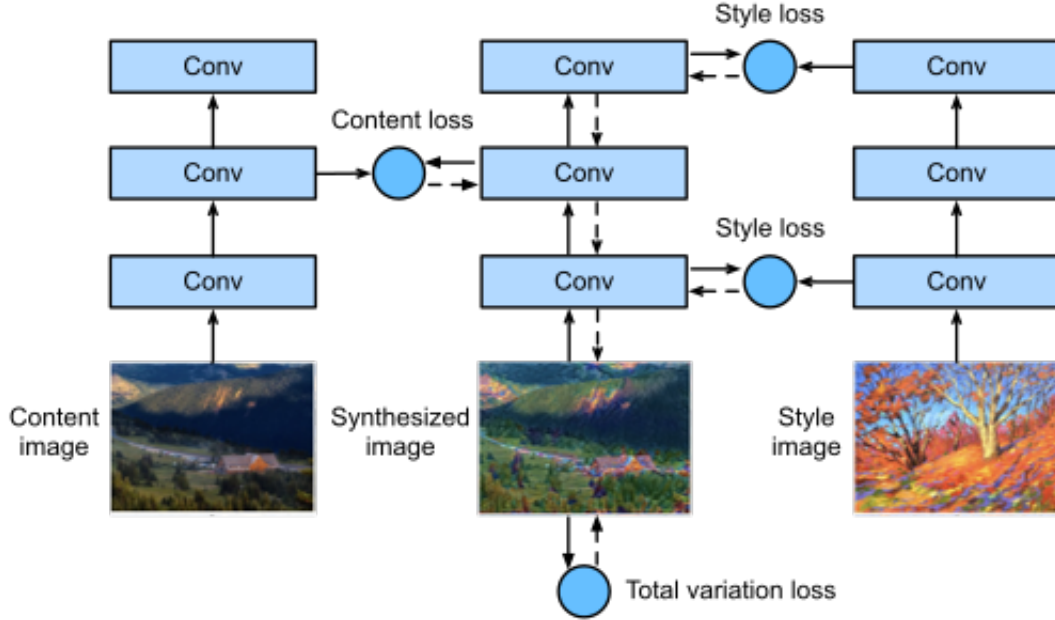


Figure 3.9: Style transfer process. Solid lines show the direction of forward propagation and dashed lines show backward propagation.

3.2.2 Extracting Features

In order to extract the content features and style features of the image, we can select the output of certain layers in the network. Generally speaking, **the closer to the input layer, the easier to extract details of the image, and the closer to the output layer, the easier to extract the global information of the image**. In order to avoid excessively retaining the details of the content image in the synthesized image, usually a layer or two closer to the output are chosen as the **content layers** to output the content features of the image. It is also common to select the output of different equally spaced layers from bottom to top of the network for extracting local and global style features. These layers are also called **style layers**.

Since there is no need to update the model parameters of the pretrained network during training, **we can extract the content and the style features even before the training starts and store them in a “cache”**, and therefore during training we only need to perform forward passes in the synthesized image, not in the content or style images.

3.2.3 Defining the Loss Function

Now we will describe the loss function for style transfer. The loss function consists of the content loss, style loss, and total variation loss. We will denote the synthesized image, the

content image and the style image by x , x^C and x^S , respectively. We will also denote by n_C the number of feature maps extracted from the content image and by n_S the number of feature maps extracted from the style image. We will denote the height of the feature map of layer r by H_r , the width of the feature map of layer r by W_r and the number of channels of the feature map of layer r by C_r . Finally, we will denote the feature map of layer r of the pretrained network as f_r .

Content Loss

The content loss measures the difference in content features between the synthesized image and the content image via the mean squared error:

$$\mathcal{L}_C = \frac{1}{n_C} \underbrace{\sum_{r=1}^{n_C}}_{\text{for each feature map}} \frac{1}{C_r H_r W_r} \underbrace{\sum_{c=1}^{C_r} \sum_{i=1}^{H_r} \sum_{j=1}^{W_r}}_{\text{for each pixel of that feature map}} \left(f_r(x)_{ijc} - f_r(x^C)_{ijc} \right)^2.$$

Style Loss

Style loss, similar to content loss, also uses the squared loss function to measure the difference in style between the synthesized image and the style image. We can transform the feature map of layer r into matrix $\mathbf{X}_r$ with C_r rows and $H_r W_r$ columns. This matrix can be thought of as the concatenation of C_r vectors $\mathbf{x}_1, \dots, \mathbf{x}_{C_r}$, each of which has a length of $H_r W_r$. Here, vector $\mathbf{x}_i$ represents the style feature of channel i for feature map r .

We define the **Gram matrix** of these vectors $\mathbf{G}_r = \mathbf{X}_r (\mathbf{X}_r)^T / (C_r H_r W_r) \in \mathbb{R}^{C_r \times C_r}$, where element x_{ij} in row i and column j is the dot product of vectors $\mathbf{x}_i$ and $\mathbf{x}_j$. It represents the correlation of the style features of channels i and j . We use this Gram matrix to represent the style output of any style layer. The style loss is:

$$\mathcal{L}_S = \frac{1}{n_S} \sum_{r=1}^{n_S} \frac{1}{C_r^2} \sum_{i=1}^{C_r} \sum_{j=1}^{C_r} \left((\mathbf{G}_r)_{ij} - (\mathbf{G}_r^S)_{ij} \right)^2.$$

Here, $\mathbf{G}_r$ refers to the Gram matrix of the r -th layer of the synthesized image and $\mathbf{G}_r^S$ to the Gram matrix of the r -th layer of the style image. A good advise is to precompute the Gram matrix on the style image, because it is always the same, to avoid computing it in each iteration of the training loop. Also, if the output value is too low, we can avoid dividing by C_r^2 .

Example: Computing the Gram matrix

Suppose that the feature map of a layer has c channels, width w and height h , and the element (i, j) of channel k is denoted as x_{kij} . Then, the matrices of each channel are:

$$\begin{pmatrix} x_{111} & \dots & x_{11w} \\ \vdots & \ddots & \vdots \\ x_{1h1} & \dots & x_{1hw} \end{pmatrix}, \dots, \begin{pmatrix} x_{c11} & \dots & x_{c1w} \\ \vdots & \ddots & \vdots \\ x_{ch1} & \dots & x_{chw} \end{pmatrix}.$$

If we flatten each matrix and concatenate them as rows:

$$\mathbf{X} = \begin{pmatrix} x_{111} & \dots & x_{11w} & \dots & x_{1h1} & \dots & x_{1hw} \\ \vdots & \ddots & \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{c11} & \dots & x_{c1w} & \dots & x_{ch1} & \dots & x_{chw} \end{pmatrix} = \begin{pmatrix} \mathbf{x}_1 \\ \vdots \\ \mathbf{x}_c \end{pmatrix}.$$

Finally, we can obtain the Gram matrix as:

$$\mathbf{G} = \frac{1}{chw} \mathbf{X} \mathbf{X}^T = \begin{pmatrix} \mathbf{x}_1 \cdot \mathbf{x}_1 & \dots & \mathbf{x}_1 \cdot \mathbf{x}_c \\ \vdots & \ddots & \vdots \\ \mathbf{x}_c \cdot \mathbf{x}_1 & \dots & \mathbf{x}_c \cdot \mathbf{x}_c \end{pmatrix}.$$

Total Variation Loss

Sometimes, the learned synthesized image has a lot of high-frequency noise, i.e., particularly bright or dark pixels. One common noise reduction method is total variation denoising. We will assume the synthesized image has C channels (usually $C = 3$) and its height and width are H and W , respectively. The total variation loss is

$$\mathcal{L}_V = \frac{1}{N} \left(\sum_{c=1}^C \sum_{i=1}^H \sum_{j=1}^{W-1} |x_{c,i,j+1} - x_{c,i,j}| + \sum_{c=1}^C \sum_{i=1}^{H-1} \sum_{j=1}^W |x_{c,i+1,j} - x_{c,i,j}| \right),$$

where $N = C(H(W-1) + (H-1)W)$ is a normalization term to divide by the total number of sums, and it makes values of neighboring pixels on the synthesized image closer.

Final Loss Function

The loss function of style transfer is the weighted sum of content loss, style loss, and total variation loss. By adjusting these weight hyperparameters, we can balance among content retention, style transfer, and noise reduction on the synthesized image, as shown in Figure 3.10. Then, the loss function is:

$$\mathcal{L} = \alpha \mathcal{L}_C + \beta \mathcal{L}_S + \gamma \mathcal{L}_V.$$

We can force that $\alpha + \beta + \gamma = 1$ to have a sense of what proportion we give to each loss, but this is optional.

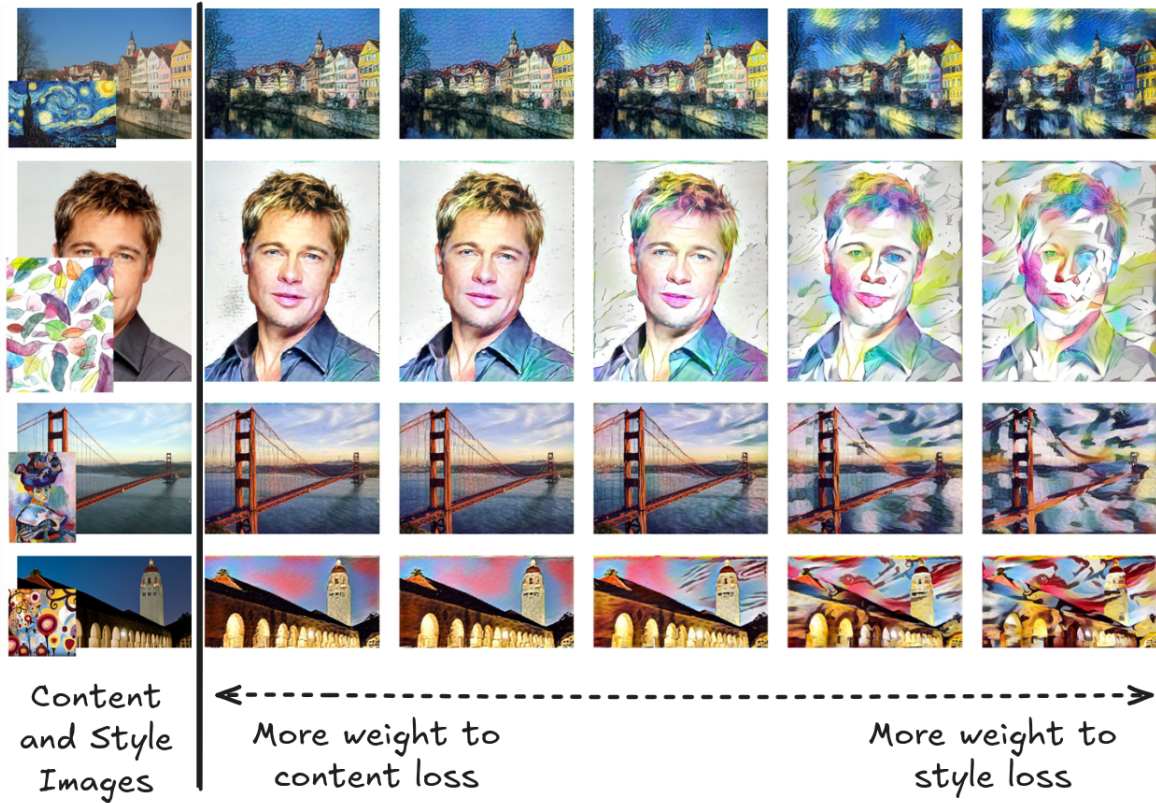


Figure 3.10: Results of using different weights in the content and style losses.

3.3 Other tasks

3.3.1 Classical Image Segmentation



Figure 3.11: Example of image segmentation based on visual similarity.

Classical or unsupervised segmentation groups pixels according to low-level properties such as color, texture, or proximity, without requiring semantic labels. For example, it may

divide a dog into one dark region containing its mouth and eyes and another region containing the rest of its body, rather than identifying the whole animal as a dog. You can see this in Figure 3.11.

3.3.2 Instance Segmentation

Instance segmentation predicts a class and a pixel mask for each countable object, or “thing,” distinguishing different instances of the same class. For example, two dogs receive separate masks. Amorphous background regions, or “stuff,” are usually outside the instance formulation.

3.3.3 Panoptic Segmentation

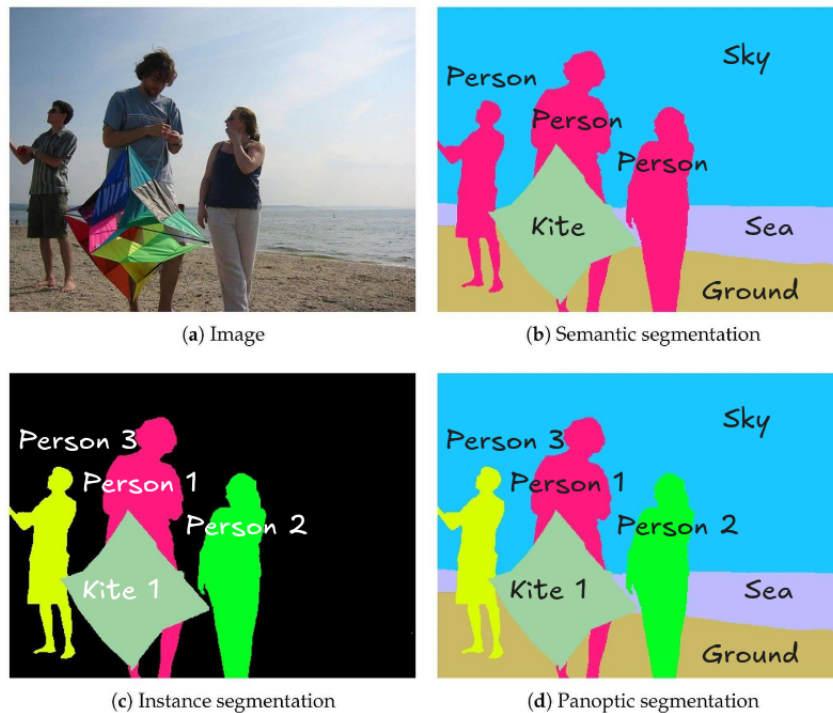


Figure 3.12: Comparison of semantic, instance, and panoptic segmentation.

Panoptic segmentation combines semantic and instance segmentation: every pixel receives a semantic class, and pixels belonging to “things” also receive an instance ID.

- Things, such as people and cars, are separated into individual object instances.
- Stuff, such as sky, grass, and road, receives semantic labels without object-instance identities, e.g., there is not a “Sea 1” and “Sea 2”.

Thus, panoptic segmentation describes both what each pixel represents and, when applicable, which object instance it belongs to.

3.3.4 Keypoint Estimation

Keypoint estimation is a computer vision task that involves detecting keypoints in objects or people within an image. These points are typically important parts or joints that describe the object's structure.

For example, in human poses, keypoints are eyes, shoulders, elbows, wrists, hips, knees, ankles, etc.; in other objects, they can be corners, edges, or points of interest.

The idea is to first detect the object (e.g., the human body) and predict the location of each keypoint as (x, y) coordinates in the image. Finally, and this is optional, we can connect the points with lines to form a skeletal representation. Models such as OpenPose, HRNet, or MediaPipe are used for this task.

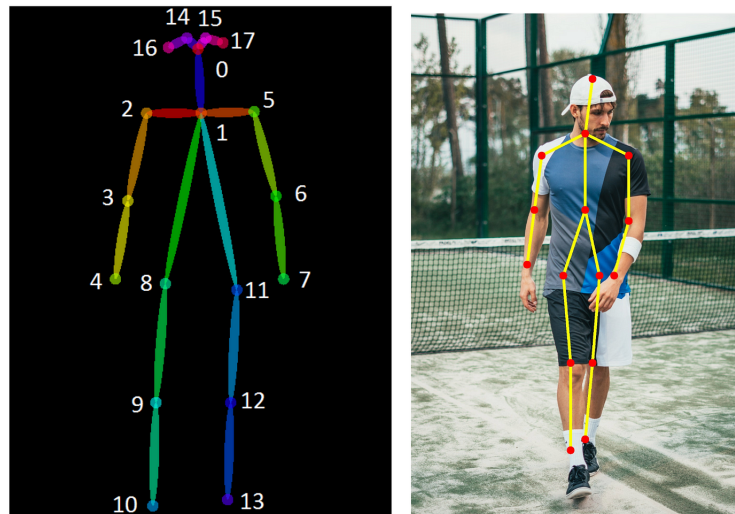


Figure 3.13: Example of keypoint estimation.

3.4 Exercises

Exercise 1: Naive Sliding-Window Segmentation

Implement a script for semantic segmentation using the naive sliding-window approach.

1. Crop patches of size $P \times P$ centered at every pixel of an input image .
2. Pass each patch through a classifier to assign a class label to the center pixel.
3. Compute the computational complexity at a high level as a function of H , W , and P . Explain why it makes this approach intractable for real-time applications.

Exercise 2: Transposed Convolution Output Shape

Given an input tensor of spatial dimensions $H_{\text{in}} \times W_{\text{in}}$, a kernel of size $k_h \times k_w$, stride s , and padding p :

1. State the general equation for computing the spatial dimensions $H_{\text{out}} \times W_{\text{out}}$ of the output feature map.
2. Calculate the output shape if $H_{\text{in}} = W_{\text{in}} = 7$, kernel size is 4×4 , stride is 2, and padding is 1.

Exercise 3: Transposed Convolution Implementation (Stride 1, Padding 0)

Write a Python/PyTorch function from scratch (using basic matrix operations, not `torch.nn.ConvTranspose2d`) that computes a 2D transposed convolution for a single-channel input $X \in \mathbb{R}^{H \times W}$ and kernel $K \in \mathbb{R}^{k \times k}$ assuming stride $s = 1$ and padding $p = 0$.

Exercise 4: General Transposed Convolution Implementation

Extend your function from the previous exercise to handle arbitrary strides $s \geq 1$, paddings $p \geq 0$, and input/output channel dimensions C_{in} and C_{out} .

Exercise 5: Mathematical Proof (Bed of Nails Initialization)

Prove that for an arbitrary 2×2 input matrix

$$M = \begin{pmatrix} a & b \\ c & d \end{pmatrix},$$

applying a transposed convolution with stride $s = 2$, padding $p = 0$, and kernel

$$K = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$$

yields an output tensor mathematically identical to the Bed of Nails upsampling operation. Show all intermediate matrix expansions step-by-step.

Exercise 6: Mathematical Proof (Nearest Neighbor Initialization)

Prove that for the same matrix M , applying a transposed convolution with stride $s = 2$, padding $p = 0$, and kernel

$$K = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$$

yields an output tensor mathematically identical to Nearest Neighbor interpolation upsampling.

Exercise 7: Traditional Upsampling Algorithms

Write a set of Python functions to upsample a feature map of shape $C \times H \times W$ to a target shape $C \times H' \times W'$ using:

1. Bed of Nails upsampling.
2. Nearest Neighbor interpolation.
3. Bilinear interpolation.
4. Max Unpooling (in this case $H = W'$ and $W = W'$).

You can include the parameters you consider for each case.

Exercise 8: Fully Convolutional Networks (FCNs)

Construct a PyTorch model for a Fully Convolutional Network:

1. Replace the classification head of a pre-trained ResNet-18 model with 1×1 convolutions to project feature channels to the number of target classes N .
2. Add transposed convolutional layers to upsample the feature map back to the original image dimensions $H \times W$.

Exercise 9: U-Net Architecture & Skip Connections

Implement a complete U-Net model from scratch in PyTorch.

1. Implement the contracting path (encoder) with standard double convolutions and max-pooling layers.
2. Implement the expanding path (decoder) with transposed convolutions and skip-connection concatenations.
3. Memory Analysis: Explain how skip connections impact CPU/GPU memory consumption during the forward-backward pass compared to an encoder-decoder architecture without skip connections.

Exercise 10: Semantic Segmentation Training

Download a standard semantic segmentation dataset (e.g., PASCAL VOC, Cityscapes, or a custom Oxford-IIIT Pet segmentation dataset). Write a complete PyTorch pipeline to train the model of the previous exercise using pixel-wise Cross-Entropy Loss.

Exercise 11: Gram Matrix

Write a Python function to compute the normalized Gram matrix for layer r $G_r \in \mathbb{R}^{C_r \times C_r}$ given an activation feature map tensor $F_r \in \mathbb{R}^{C_r \times H_r \times W_r}$. Prove mathematically that entry (i, j) of G_r represents the cosine similarity between flattened feature channels i and j if the feature vectors are normalized to unit magnitude.

Exercise 12: Style Transfer Loss

Create separate, modular functions to evaluate the following style transfer loss terms:

1. Content Loss $\mathcal{L}_C$ between synthesized features and target content features across multiple selected content layers.
2. Style Loss $\mathcal{L}_S$ using Gram matrices across multiple selected style layers.
3. Total Variation Loss $\mathcal{L}_V$ to penalize spatial high-frequency artifacts in the synthesized image.

Exercise 13: Style Transfer Training

Implement the complete Neural Style Transfer algorithm using a pre-trained VGG-19 network backbone:

1. Freeze the VGG-19 backbone parameters.
2. Pre-calculate and cache target content and style representations.
3. Instantiate a trainable image tensor x initialized as the content image x^C .
4. Optimize x directly using Adam optimizer for 200 iterations.

Exercise 14: Style Transfer Loss Impact

Run experiments on your style transfer pipeline under the following conditions and document your observations:

1. Vary the hyperparameter ratio $\alpha/\beta \in \{10^{-4}, 10^{-2}, 1, 10^2, 10^4\}$.
2. Initialize x with pure Gaussian white noise versus initializing with the content image x^C .
3. Set the total variation weight $\gamma = 0$ versus $\gamma = 10^{-3}$. How does this affect spatial smoothness?

Exercise 15: Comparative Analysis of Vision Tasks

Create a structured comparison table analyzing the input requirements, output representations, target loss functions, and core application domains for the following computer vision paradigms:

1. Semantic Segmentation
2. Instance Segmentation
3. Panoptic Segmentation
4. Keypoint Estimation / Pose Estimation

